Lazy Symbolic Model Checking

Jin Yang
Strategic CAD Labs
Design Technology, Intel Corp.
jinyang@ichips.intel.com

Andreas Tiemeyer
Design Technology, Intel Corp.
andreas.tiemeyer@intel.com

Abstract

In this paper we present a lazy model checking approach aimed at improving the efficiency and capacity of symbolic model checking. The lazy approach dynamically computes an abstraction of a circuit model for each pre-image computation based on the partial result leading to the computation. At the heart of the approach is a lazy algorithm for transition relation building and pre-image computation. A variable minimization heuristic is then proposed to maximize the benefit of the lazy algorithm in iterative fix-point computations. This approach shows greater promise in complexity reduction than the cone of influence approach.

1. Introduction

Symbolic model checking reduces verification complexity by capturing the regularity in the state space of circuit models using binary decision diagrams (BDD's) [3]. In 1987, McMillan [12] implemented a BDD-based algorithm SMV for CTL model checking [6]. Since then many techniques [7] have been proposed to improve the performance and capacity of the algorithm.

One very well known technique is the *cone of influence* (COI) reduction [9, 5, 7]. The basic idea is to build a dependency graph for a circuit model such that variable v depends on variable w if w appears in the next state function of v. Given a specification, an abstraction of the model is built that only consists of the next state functions for the variables in the dependency closure of the variables in the specification. There are two drawbacks. First, the fact that a variable is in the closure does not necessarily mean that it truly influences the specification. If it does not, then it could be eliminated from the abstraction. Second, the abstract model, once built, is used in its entirety in every model checking step, although only a portion of it is useful for any particular step. Several iterative model checking algorithms have improved upon the reduction technique by starting from a small portion of the dependency closure. When model checking fails to produce a satisfactory answer, they extend the abstract model by adding more variables from the closure. This process is repeated until a definite yes/no answer is produced [1, 9, 10, 11]. While the first drawback has been addressed by these algorithms, the second drawback remains unresolved.

In this paper, we present a lazy model checking approach that simultaneously solves the two problems by closely tying the abstraction of a model to each pre-image computation in the model checking process. First, we propose a lazy algorithm for pre-image computation and transition relation building (Section 1.1). When computing the pre-image of a predicate, the algorithm builds an abstract transition relation that is composed from the next state functions for the variables in the predicate only. The next state function for a variable is built and composed into the transition relation only at the very first time it is involved in a pre-image computation. Second, we show how to apply the lazy pre-image algorithm effectively in fix-point computations (Section 2). The key idea is a variable minimization heuristic for the frontiers in a fix-point computation based on the exploitation of the *dontcare* space. We have incorporated this lazy approach into SMV, and have been able to verify some difficult properties that resisted verification using the COI reduction approach (Section 3).

A related approach was proposed in [2] for the SAT based CMU Bounded Model Checker. The *bounded cone of influence* is an extension of the COI reduction. The key idea is to include the next function for a variable at depth k of the unrolled model only if the variable is k steps away in the dependency graph from the variables in the specification. However, this approach is specific to the SAT based algorithm. Furthermore, a variable involved at depth k may still not be truly influential.

1.1 Lazy Pre-Image Computation and Transition Relation Building

Let us first define a simple circuit model. A *circuit model* is a tuple $C = (V = V_S \cup V_I, \mathcal{N})$, where (1) V_S and V_I are, respectively, a set of boolean *internal variables* and a set of boolean *input variables*, and (2) $\mathcal{N}$ is a set consisting of a *next state function* $N_v : V \rightarrow \{0,1\}$ for each $v \in V_S$. Given a partitioning $\{V_1, V_2, \ldots, V_k\}$ of the internal variables in V_S, the *partitioned transition relation* for the model is the following implicitly conjoined form:

$$R(V, V') = R_1(V, V_1') \wedge R_2(V, V_2') \ldots \wedge R_k(V, V_k')$$

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
DAC 2000, Los Angeles, California
©2000 ACM 1-58113-187-9/00/0006..$5.00

Authorized licensed use limited to: Nanjing University. Downloaded on July 04,2021 at 16:01:27 UTC from IEEE Xplore. Restrictions apply.

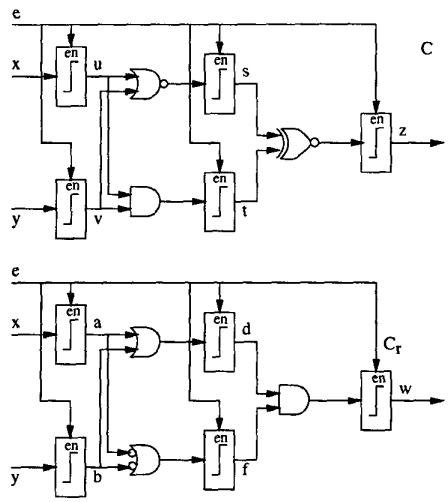

Figure 1: A Circuit Example

where the i-th partition is $R_i(V, V_i') = \bigwedge_{v' \in V_i'}(v' = N_v)$ and V_i' (V') is the set of next variables for V_i (V). We shall call $v' = N_v$ the *next state relation* for v.

Using the partitioned transition relation, the *pre-image* of a state predicate $S(V)$, denoted by $pre(S(V))$, can be computed using the following relational product algorithm utilizing early variable quantification ([4, 8]):

$$\begin{aligned} pre(S(V)) \;=\; & \exists V_1'.[R_1(V, V_1') \wedge (\exists V_2'.[R_2(V, V_2') \wedge \\ & (\ldots [R_k(V, V_k') \wedge (\exists V_I'.S(V'))] \ldots)])] \end{aligned}$$

where $S(V')$ is obtained from $S(V)$ by substituting each variable $v \in V$ with its next version $v' \in V'$.

A drawback with this algorithm is that it involves the entire partitioned transition relation every time it is invoked, even if $S(V)$ may only contain a few variables. Obviously, not every next state relation is relevant to a particular invocation. The following theorem shows how to build an abstraction of the model for each invocation.

THEOREM 1. *The pre-image of predicate $S(W)$ for some $W \subseteq V$ can be computed as*

$$pre(S(W)) \;=\; \exists W'.[\wedge_{v' \in W_S'}(v' = N_v) \wedge S(W')],$$

where W_S' are the set of next internal variables in W'.

Due to the page limitation, we omit the proof of the theorem. Although in the same spirit, the abstraction approach provided by the theorem is different from the COI reduction [7]. Instead of using a static abstraction derived from the specification throughout the model checking process, our approach provides a different abstraction for every pre-image computation based on the predicate involved, and can potentially lead to greater efficiency and capacity improvement in model checking. Let us further examine the difference using the model example in Figure 1, which is composed of two circuits C and C_r.

Suppose we want to perform an equivalence checking on the two outputs z and w, i.e., checking $AG(z = w)$. The COI reduction will pull in the entire model and use it in every pre-image computation. However, when computing $pre(z = w)$ in the first iterative step of the equivalence checking, only the next state relations for z and w are needed.

Algorithm 1: *LazyPreImage*($S(W)$)

1. W_S' := next internal variables in W';
   W_I' := next input variables in W';
2. **for** each $w' \in W_S'$
3. **if** N_w has not been evaluated
4. conjunct $w' = N_w$ to the partition including w';
    endif
   endfor
5. $pre := \exists W_I'.S(W')$;
6. **for** each i from k downto 1
7. **if** $V_i' \cap W' \neq \emptyset$
8. $R_a := \exists(V_i' \setminus W').R_i(V, V_i')$;
9. $pre := \exists V_i'.[R_a \wedge pre]$;
    endif
   endfor
10. return *pre*;

end.

Figure 2: Lazy Pre-Image Computation

The complete lazy algorithm for transition relation building and pre-image computation is listed in Figure 2. At the beginning of the model checking process, $R_i(V, V_i') = 1$ for all $1 \leq i \leq k$. There are three places in the algorithm where the laziness is utilized. First, each transition relation partition is built lazily and on-demand (Steps 3-4). This is especially important for a BDD based approach where a next state function for an internal variable is implicitly but efficiently encoded and the BDD for the function has to be explicitly built. Second, only influential partitions are selected (Step 7). Finally, all next variables that are not in W' are existentially quantified out from each partition before it is used in the computation (Step 8).

2. Laziness in Fix-point Computation

In this section, we explore how to effectively apply the lazy algorithm to fix-point computations. Let us consider the fix-point algorithm for computing $E[f\ U\ g]$. A naive application of the lazy algorithm will yield the fix-point algorithm in Figure 3, where F and G are, respectively, the state predicates for formulas f and g. In each iteration of the algorithm, *front* is conjoined with $\neg Y$ in order to exclude the states whose pre-images have already been computed. As a consequence, the number of variables in *front* may grow from one iteration to the next and thus render the lazy algorithm less and less effective.

To overcome this problem, we want to reduce the number of variables in *front* by exploring the *dontcare* space. More specifically, we find a new predicate *Front* such that (1) $(front \Rightarrow Front) \wedge (Front \Rightarrow Y)$, and (2) the number of variables in *Front* is minimized. Condition (1) can be slightly re-phrased as $(front \Rightarrow Front) \wedge (Front \wedge \neg Y = 0)$.

Authorized licensed use limited to: Nanjing University. Downloaded on July 04,2021 at 16:01:27 UTC from IEEE Xplore. Restrictions apply.

```
Algorithm: EU(F, G)

1. Y := G;
2. front := G;
3. while(front != 0)
4.        pre := F ∧ LazyPreImage(front);
5.        front := pre ∧ ¬Y;
6.        Y := front ∨ Y;
   endwhile
7. return Y; end.
```

Figure 3: Lazy Fix-point Computation

Given that we already know $front \wedge \neg Y = 0$, the following lemma suggests several ways to eliminate a variable from *front*. Due to the space limitation, we shall omit the proofs of these lemmas. Given a predicate S, let $S[c/v]$ denote the predicate obtained from S by substituting variable v with boolean constant c.

LEMMA 1. *Let v be a variable in either front or $\neg Y$. Then*

1. *If v is in front only, $(\exists v.front) \wedge \neg Y = 0$.*
2. *If v is in both predicates, then $(\exists v.front) \wedge \neg Y = 0$, iff*

   $$front[1/v] \wedge (\neg Y)[0/v] = 0 \wedge front[0/v] \wedge (\neg Y)[1/v] = 0.$$

3. *If v is in $\neg Y$ only, then for any variable w in front, $(\exists w.front) \wedge \neg Y = 0$, iff*

   $$(\exists w.front) \wedge ((\neg Y)[1/v] \vee (\neg Y)[0/v]) = 0.$$

Based on the lemma, we propose a BDD-based greedy variable reduction algorithm (Figure 4), assuming the order of variables is fixed. Steps 9-10 in the variable reduction algorithm implement (1), Steps 11-19 implement (2), and Steps 20-21 implement (3). To avoid computing the same function twice, Steps 5-6 and Step 22 use a BDD hash table to store (*Insert*) and retrieve (*LookUp*) the computed results. Function *SatisfyAnd* is a BDD function that checks if two predicates intersect. To reduce the number of variables in *front* using this algorithm, we invoke $RedVar(front, \neg Y)$ after Step 6 in Figure 3.

Let us look at an example to illustrate the usage of the algorithm. Suppose we want to compute $AG(z = w)$ for the model in Figure 1. After the first iteration of $EU(1, (z \neq w))$ in Figure 3, we have:

$$\begin{aligned} front &= e \wedge (z = w) \wedge ((s = t) \neq (d \wedge f)), \\ Y &= (z \neq w) \vee e \wedge ((s = t) \neq (d \wedge f)). \end{aligned}$$

Both predicates contain e, s, t, d, f as well as z and w. If we apply the reduction algorithm in Figure 4 to *front*, assuming the variables are ordered as z, w, s, t, d, f and e, then we have $Front = e \wedge ((s = t) \neq (d \wedge f))$. Since z and w are eliminated, the next pre-image computation will naturally focus on the middle stage of the pipeline.

```
Algorithm 2: RedVar(front, Ybar)

1. if front is a boolean constant
2.    return front;
3. elsif Ybar is a boolean constant
4.    return 1;
5. elsif LookUp(RedVar, front, Ybar) is not empty
6.    return LookUp(RedVar, front, Ybar);
   endif
7. Let v be the top level variable in front;
8. Let w be the top level variable in Ybar;
9. if level(v) < level(w)
10.   res := RedVar(front[1/v] ∨ front[0/v], Ybar);
11. elsif level(v) = level(w)
12.   front_1 := RedVar(front[1/v], Ybar[1/v] ∨ Ybar[0/v]);
13.   front_0 := RedVar(front[0/v], Ybar[1/v] ∨ Ybar[0/v]);
14.   if front_1 = front_0
15.      res := front_1;
16.   elsif SatisfyAnd(front_1, Ybar[0/v]) ≠ 0 or
            SatisfyAnd(front_0, Ybar[1/v]) ≠ 0
17.      res := v ∧ front_1 ∨ v̄ ∧ front_0;
18.   else
19.      res := front_1 ∨ front_0;
      endif
20. else
21.   res := RedVar(front, Ybar[1/w] ∨ Ybar[0/w]);
   endif
22. Insert(RedVar, front, Ybar, res);
23. return res;
end.
```

Figure 4: Variable Reduction

We have developed a similar lazy fix-point algorithm for computing $A[f\ U\ g]$ under a set of fairness constraints $\mathcal{F}$. The algorithm exploits the fact that in a data-path circuit, both f and $\mathcal{F}$ are most likely to be in the control part.

Finally, our lazy approach has some additional benefits. First, if the bin packing approach is used to partition a circuit, the laziness of building the partitioned transition relation will provide a natural way of clustering variables that are logically close together. Second, the on-the-fly declaration of BDD variables during the next state function computation will offer a natural variable ordering.

3. Experimental Results

We have incorporated our approach into SMV and tested it on several Intel circuits. We first abstracted the circuits using the COI reduction and then ran them through SMV with and without the lazy option. We used the backward search only evaluation strategy in SMV. The experiments were done on an HP workstation with 256MB memory, and the comparison result is shown in Table 1. The first two columns show the alias and the number of state variables in a circuit.

The first two examples are from the same data-path unit with complex control. For ckt1, we verified a complex CTL liveness property which states that data never gets lost in

Authorized licensed use limited to: Nanjing University. Downloaded on July 04,2021 at 16:01:27 UTC from IEEE Xplore. Restrictions apply.

Circuit after COI reduction		SMV		Lazy-SMV	
name	#variables	time(sec.)	memory(MB)	time(sec.)	memory(MB)
ckt1	2797	Out. Mem.	Out. Mem.	508	3.9
ckt2	2790	Out. Mem.	Out. Mem.	544	3.8
ckt3	380	Unfinished	Unfinished	19	6.4
ckt4	239	Unfinished	Unfinished	1,000	1.8
ckt5	175	1043	7.2	14683	37
ckt6	129	613	25.8	29	5.0
ckt7	106	792	15.3	20	6.4
ckt8	103	177	20.2	174	14.5

Table 1: Comparison of SMV and Lazy-SMV

the pipeline. For ckt2, we verified in a single run 46 **mutex** properties embedded in the unit, some of which compare parts of the unit against small embedded reference models. Both ckt1 and ckt2 were the result of applying the basic COI reduction on the original unit, and still contain ≥ 2000 variables. In both examples, Lazy-SMV completed within 10 minutes and required small amounts of memory.

In each of the other six examples, an LTL property was compiled into a finite state machine and compared against a circuit to check if they always agree. The circuit was abstracted from a unit using a sophisticated COI reduction that allows a signal to be freed or set to a constant value. In ckt3 and ckt4, Lazy-SMV completed but SMV failed. In ckt6, ckt7 and ckt8, Lazy-SMV compared favorably in both time and memory to SMV. In ckt5, however, Lazy-SMV fared poorly. This was primarily caused by a different BDD reordering process triggered by the lazy approach. The overhead and the ineffectiveness of the *RedVar* algorithm for this example also played a factor.

One conclusion we can draw from the experiments is that Lazy-SMV is likely to be more effective for the class of properties in which the structure, temporal flow and assumption information of the property are explicitly preserved.

4. Conclusion

In this paper, we presented a lazy model checking approach. By dynamically building the optimal abstraction for each pre-image computation in a lazy fashion, this approach can make symbolic model checking more efficient. Our approach should also blend well with other symbolic techniques due to its tight integration to the pre-image computation. Further improvements are desired. To just name a few, the lazy algorithm can be applied to approximate forward reachability analysis by identifying the next variables that depend on the variables in a given state predicate. The variable reduction heuristic can be improved by finding out a way to call $SatisfyAnd$ less in its recursive procedure.

Acknowledgment.
We would like to thank Carl Seger, Brian Moore and Yatin Hoskote for reviewing the paper and providing valuable suggestions.

5. References

[1] F. Balarin and A. Sangiovanni-Vincentelli. An iterative approach to language containment. In *Proc. of the 5th Workshop on CAV*, pages 193–195, 1999.

[2] A. Biere, E. Clarke, R. Raimi, and Y. Zhu. Verifying safety properties of a PowerPC^{TM} microprocess using symbolic model checking without BDDs. In *CAV'99*, 1999.

[3] R. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Trans. on Computers*, C-35(8):677–691, August 1986.

[4] J. Burch, M. Clarke, K. McMillan, and D. Dill. Sequential circuit verification using symbolic model checking. In *DAC'99*, 1990.

[5] W. Chan, R. Anderson, P. Beame, and D. Notkin. Improving efficiency of symbolic model checking for state-based system requirements. In *Proc. of the ACM ISSTA-98*, 1998.

[6] E. Clarke, E. Emerson, and A. Sistla. Automatic verification of finite-state concurrent systems using temporal logic specifications: A practical approach. In *Proc. of the 10th ACM Symposium on Principles of Programming Languages*, pages 117–126, 1983.

[7] E. Clarke, O. Grumberg, and D. Peled. *Model Checking.* The MIT Press, 1999.

[8] R. Hojati, S. Krishnan, and R. Brayton. Early quantification and partitioned transition relations. In *Proc. of the 1996 ICCAD*, pages 12–19, 1996.

[9] R. Kurshan. *Computer-Aided Verification of Coordinating Processes: The Automata-Theoretic Approach*, pages 170–172. Princeton University Press, 1994.

[10] W. Lee, A. Pardo, J.-Y. Jang, G. Hachtel, and F. Somenzi. Tearing based automatic abstraction in CTL model checking. In *Proc. of the IEEE/ACM ICCAD*, pages 76–815, 1996.

[11] J. Lind-Nielsen and H. Andersen. Stepwise CTL model checking of state/event systems. In *CAV'99*, 1999.

[12] K. McMillan. *Symbolic Model Checking: An Approach to the State Explosion Problem.* PhD thesis, School of Computer Science, Carnegie Mellon University, 1992.

Authorized licensed use limited to: Nanjing University. Downloaded on July 04,2021 at 16:01:27 UTC from IEEE Xplore. Restrictions apply.